

**Keywords:** program synthesis • neural functions • adapter fine-tuning • natural language specifications

# Compile by Training: Turning Natural-Language Specifications into Local Neural Functions

*Teacher LLMs Generate Task-Specific Training Data at Compile Time; a Lightweight Adapter Runs Offline at 83.6% Semantic Accuracy Where Fast Compilers Achieve Zero*

By Yuntian Deng, Pengyu Nie, Stuart Shieber (Harvard University; et al.)

arXiv:2609.04199 • EMNLP 2026 System Demonstrations

A natural-language specification — “given an address string, return the city component, normalised to title case, with abbreviations expanded” — describes a deterministic function. Current practice runs a large frontier LLM at inference time for every query, incurring repeated cost, requiring internet connectivity, and providing no persistent, versioned artifact. Compile by Training (CbT) converts the specification into a lightweight adapter that runs locally on a compact interpreter model, with no runtime teacher access. At compile time, teacher LLMs generate diverse input-output examples that train the adapter to implement the specification exactly. The result runs at 12 ms per query with 83.6% semantic accuracy on FuzzyBench-Hard — a benchmark where the Program-as-Weights fast compiler achieves zero exact matches.

## AT A GLANCE

**Problem:** Repeating a frontier LLM call for every invocation of a fixed natural-language-specified task is expensive, slow, and requires cloud connectivity.

**Why it matters:** Many deployed NLP tasks are static, reused repeatedly, and need local, versioned, composable execution.

**Method:** Teacher LLMs enumerate examples; compact adapter is trained on them; runs offline at inference.

**Result:** 83.6% semantic accuracy on FuzzyBench-Hard (PAW: 0% exact); 12 ms/query local runtime; composes and versions like code.

**Evidence:** FuzzyBench-Hard, WebTask, AvatarControl; ablations on committee size, training set size, LoRA rank.

**Motivation: From Repeated Inference to Compiled Functions**

For a task that will be called millions of times — normalising product names, extracting structured fields from documents, controlling a 3D avatar from natural language — the cost of a frontier LLM call per invocation is prohibitive. The output distribution of the task is fixed by its specification; there is no need to re-solve it at runtime.

The Program-as-Weights (PAW) framework already embeds this intuition: a specification is treated as a “program” compiled into a neural network’s weights. PAW’s fast compiler generates examples in seconds and fine-tunes a small adapter. But fast compilation is shallow — too few examples, too little training — and fails on hard specifications.

CbT extends PAW with a deeper, more expensive compilation that produces far higher accuracy while preserving the local, fast, offline runtime.

## Compile by Training: The Four Steps

**Step 1: Example generation by teacher committee.** Given specification  $s$ , a committee of  $K$  teacher LLMs is queried to generate diverse  $(x_i, y_i^*)$  input-output pairs. Multiple teachers reduce individual model errors.

**Step 2: Validation by cross-reference.** An input  $x$  is accepted with output  $y^*$  if at least  $k$  of  $K$  teachers agree on  $y^*$ . Disagreed pairs are discarded. The disagreement rate  $d$  serves as a specification ambiguity signal.

**Step 3: Adapter training on curated examples.** A LoRA adapter  $\delta\theta$  for a compact local interpreter is trained on the validated dataset  $\mathcal{D}_s = \{(x_i, y_i^*)\}$ :

$$\mathcal{L}_{\text{CbT}} = - \sum_{(x, y^*) \in \mathcal{D}_s} \log p_{\theta + \delta\theta}(y^* | s, x)$$

**Step 4: Local deployment.** The compiled artifact  $(s, \delta\theta)$  is stored. At inference time, the compact model with its adapter processes new inputs locally, without any teacher or internet access, at 12 ms per query.

## Semantic Accuracy Metric

Standard exact-match fails for tasks with multiple valid output forms. CbT uses **semantic accuracy**:

$$\text{SemAcc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{sem-equiv}(f_\theta(x_i), y_i^*)]$$

where sem-equiv judges equivalence using a judge LLM, accepting different surface forms of the correct answer.

## Accuracy vs. Compile Compute

Empirically, accuracy saturates as a function of training set size:

$$\text{SemAcc}(N_{\text{ex}}) \approx A - B \cdot e^{-\alpha N_{\text{ex}}}$$

with  $A \approx 0.85$ ,  $B \approx 0.84$ ,  $\alpha \approx 0.003$  on FuzzyBench-Hard. Saturation occurs around  $N_{\text{ex}} = 1000$  examples, corresponding to a compile time of approximately one minute (GPT-4o teacher, parallel calls).

## Composition

CbT functions compose by chaining. If  $f_A$  implements specification  $s_A$  and  $f_B$  implements  $s_B$ , a composed function  $f_{A \circ B}$  is compiled by: (1) generating a composed specification  $s_{A \circ B}$ ; (2) using  $f_A$  to produce intermediate inputs for  $f_B$  as training examples. The resulting adapter implements  $s_{A \circ B}$  locally without the intermediate function at inference time.

## Experimental Results

| Method                   | SemAcc       | Compile     | Runtime        |
|--------------------------|--------------|-------------|----------------|
| Teacher (GPT-4o) runtime | 91.2%        | 0 s         | 2.1 s/q        |
| PAW fast compiler        | 51.3%        | 8 s         | 12 ms/q        |
| <b>CbT (ours)</b>        | <b>83.6%</b> | <b>60 s</b> | <b>12 ms/q</b> |
| CbT + validation         | 87.1%        | 120 s       | 12 ms/q        |

CbT closes 80% of the quality gap between PAW and teacher-at-runtime while matching PAW’s fast local inference. On WebTask, two-function compositions achieve 76.4% task success, confirming composition viability.

## Ablations

**Committee size:** 3 teachers reduces disagreement from 31% (1 teacher) to 12% and improves SemAcc from 79.4% to 83.6%.

**Training set size:** 1000 examples saturates; 500 achieve 80.1%; 2000 add only 0.8 points.

**LoRA rank:** Rank 16 achieves full accuracy; rank 4 loses 2.1 points; rank 64 adds nothing.

**Specification clarity:** Disagreement rate  $d > 0.4$  predicts low SemAcc; prompting users to clarify ambiguous specifications recovers 4.3 SemAcc points on average.

**Practical applications:** Website form-filling helpers and language-controlled 3D avatar motion demos confirm that CbT applies to realistic end-to-end tasks beyond benchmarks.

## Reference

Yuntian Deng, Pengyu Nie, and Stuart Shieber. **Compile by Training: Turning Natural-Language Specifica-**

**tions into Local Neural Functions.** arXiv:2609.04199, EMNLP 2026 System Demonstrations. <https://arxiv.org/abs/2609.04199>